Nom et prénom : _____ Note : _____

1 Contexte

Un **WAF** pour **W**eb **A**pplication **F**irewall est un logiciel faisant office de pare-feu au niveau de la couche applicative du protocol **HTTP**. Le fonctionnement repose sur l'analyse de logs des requêtes qui arrivent sur un service web et l'application de règles pour déterminer si le trafic semble suspect ou non. Si le trafic est considéré suspect, l'adresse IP à l'origine de la requête doit être mise sur une liste noire pour tenter d'interrompre une potentielle attaque.

L'objectif de ce TP est de réaliser un script Python mettant en place les bases de ce type d'application (inspecter un fichier de log, déterminer si le trafic est suspect, bannir l'adresse IP source).

Les fichiers à votre disposition sont :

— `http_requests.log` contenant un échantillon de logs
— `main.py` contenant une fonction pour extraire les données d'une ligne de log

2 Exemple

Voici un exemple de logs à traiter (les dates et heures sont tronquées) :

```
192.168.1.10 - [12/Feb/2026:09:00:14] "GET / HTTP/1.1" 200 512 "-" "Mozilla/5.0"
192.168.1.11 - [...] "GET /products HTTP/1.1" 200 2048 "-" "Mozilla/5.0"
192.168.1.12 - [...] "GET /product/42 HTTP/1.1" 200 1024 "-" "Mozilla/5.0"
192.168.1.13 - [...] "POST /login HTTP/1.1" 401 256 "-" "Mozilla/5.0"
192.168.1.13 - [...] "POST /login HTTP/1.1" 401 256 "-" "Mozilla/5.0"
192.168.1.13 - [...] "POST /login HTTP/1.1" 401 256 "-" "Mozilla/5.0"
192.168.1.13 - [...] "POST /login HTTP/1.1" 401 256 "-" "Mozilla/5.0"
192.168.1.13 - [...] "POST /login HTTP/1.1" 401 256 "-" "Mozilla/5.0"
45.83.12.4 - [...] "GET /index.php?id=1%20OR%201=1 HTTP/1.1" 500
45.83.12.4 - [...] "GET /index.php?id=2%20UNION%20SELECT%20password HTTP/1.1" 500
```

Dans cet exemple les IPs `192.168.1.13` et `45.83.12.4` doivent être bannies pour les raisons suivantes :

— Trop de requêtes suspectes détectées sur `/login`
— Tentatives d'injection SQL détectées sur `index.php`

3 Etapes

Vous devez faire valider chaque étape avant de passer à la suivante.

1. (6 points) Parsing des logs

   Objectif : Extraire les informations utiles de chaque ligne du fichier `http_requests.log`.

   Étapes :
   - Initialisez un dictionnaire `suspiciousRequests` qui contiendra pour chaque IP le nombre de requêtes suspectes détectées.
   - Ouvrez le fichier de log `http_requests.log` en **mode lecture**
   - Pour chaque ligne du fichier de logs, utilisez la fonction `parse_log_line` afin de récupérer un **dictionnaire** contenant les informations extraites de la requête.
   - Ce dictionnaire contient notamment :
     - L'**IP source**
     - La **date et l'heure**
     - La **requête HTTP**
     - Un booléen indiquant si la requête est **suspecte**
   - Si la requête est considérée comme suspecte :
     - Si l'IP n'est pas présente dans le dictionnaire `suspiciousRequests`, il faut l'ajouter et initialiser le compteur à 1.
     - Si l'IP est présente dans le dictionnaire, il faut incrémenter le nombre de requêtes suspectes.

   Note : `parse_log_line` retourne les informations sous forme de **dictionnaire**.

   Exemple de dictionnaire a générer :

   ```
   suspiciousRequests = {"192.168.1.100" : 6, "192.168.1.102" : 1}
   ```

2. (3 points) Détection des IP à bannir

   Objectif : déterminer quelles IP dépassent le **seuil de requêtes suspectes** (`threshold = 4`).

   Étapes :
   - Créez un **ensemble** (`set`) `toBanIps` qui contiendra les IP à bannir.
   - Parcourez les **entrées** du dictionnaire `suspiciousRequests`.
   - Si une IP a un nombre de requêtes suspectes **supérieur au seuil**, elle est ajoutée à `toBanIps`.

3. (3 points) Génération de la liste noire

   Objectif : écrire les IP à bannir dans un fichier `blacklist`.

   Étapes :
   - Ouvrez un fichier nommé `blacklist` en **mode ajout**
   - Pour chaque IP de `toBanIps`, écrire une nouvelle ligne dans le fichier.

   Exemple de contenu final :

   ```
   192.168.1.100
   192.168.1.102
   ```

4. (4 points) Configuration du programme

   Objectif : configurer le programme à l'aide d'un fichier au format `ini` ou `json`.

   Étapes :
   - Au démarrage du programme, vérifier si un fichier `config.json` ou `config.ini` existe dans le même dossier que le programme.
   - Ce fichier doit être chargé dans un dictionnaire (`dict`), chaque clé du dictionnaire correspond à un paramètre modifiable :
     - Le seuil (`threshold`)
     - Le chemin d'export de la liste noire
   - Adapter le reste du code pour utiliser les valeurs présentes dans le dictionnaire de configuration.
   - Le fichier de configuration étant optionnel, le programme ne doit pas lever d'erreur s'il n'est pas présent et des valeurs par défaut adaptées doivent être utilisées.

   Astuce : vous pouvez vous aider des modules :
   - `https://docs.python.org/fr/3.10/library/json.html#module-json`
   - `https://docs.python.org/fr/3.10/library/configparser.html`

   Exemple de fichier de configuration :

   ```
   {
     "threshold": 4,
     "blacklist_file_path": "/tmp/blacklist.txt"
   }
   ```

5. (4 points) Amélioration au choix
   - Vérifier le nombre de requêtes suspectes sur une plage de temps
   - Détecter les requêtes trop rapprochées
   - Produire, en plus du fichier liste noire, un fichier de log contenant la raison du banissement d'une IP